

# Hyperparameter Optimization with Optuna

Welcome to the Optuna-based hyperparameter optimization tutorial! In this interactive notebook, you will explore world of hyperparameter tuning for a Convolutional Neural Network (CNN) specifically aimed at image classification using the CIFAR-10 dataset. Hyperparameter optimization is pivotal in enhancing model performance, making your models more accurate and efficient.

Optuna, a robust and versatile library, plays a central role in automating and streamlining this process. It empowers you to navigate through complex hyperparameter spaces with ease. In this tutorial, you will engage with Optuna's core functionalities, and you'll also have the opportunity to construct a flexible CNN architecture. This adaptable design is essential for understanding how models can be fine-tuned effortlessly to suit various hyperparameter configurations.

Throughout this session, you will:

- Learn how to set up and execute an Optuna study, incorporating all essential elements required for effective hyperparameter optimization.
- Perform a thorough analysis of the results to evaluate how different hyperparameters influence model performance, gaining insights into their practical impact.

Additionally, this tutorial includes an optional section where you will compare two prevalent methods of hyperparameter optimization: Optuna's default sampling method (Tree-structured Parzen Estimator, or TPE) and the traditional Grid Search method. This comparison will not only highlight the strengths of Optuna but also provide a clearer perspective on how it can outperform conventional optimization techniques.

## Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import optuna
import matplotlib.pyplot as plt
import helper_utils
import torch.nn.functional as F
from pprint import pprint

helper_utils.set_seed(15)

# Check device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(device)

cuda
```

# Hyperparameter Optimization for CNNs on CIFAR-10

In this section, you explore the vital task of finding the optimal hyperparameters for a Convolutional Neural Network (CNN) tailored to the CIFAR-10 dataset. Utilizing Optuna, a sophisticated framework for hyperparameter optimization, your goal is to streamline and automate the process, ensuring efficiency and effectiveness. The selection of hyperparameters is notably intensive computationally and depends on various factors including the architecture of the model, the dataset characteristics, and the specific training processes involved. These elements, collectively and individually, have significant impacts on the performance outcomes of the model.

## Defining a Flexible CNN Architecture

The model architecture here is deliberately designed to be flexible, accommodating variability in its layers which is pivotal for adapting to different hyperparameter configurations suggested by Optuna during optimization trials.

The architecture is defined in a modular manner, allowing for easy adjustments and experimentation with different layer configurations, activation functions, and other hyperparameters.

`FlexibleCNN` is a class that encapsulates the architecture of the CNN model:

- `__init__`: The constructor initializes the model's feature extraction layers.
  - It constructs a series of convolutional blocks based on the `n_layers` parameter. Each block is a sequence of `nn.Conv2d`, `nn.ReLU`, and `nn.MaxPool2d`.
  - The `in_channels` for each block is set to the `out_channels` of the preceding block to ensure a seamless data flow.
  - All blocks are combined into a single `nn.Sequential` module assigned to the `.features` attribute, which handles feature extraction.
  - The classifier, `.classifier`, is initially set to `None` and will be constructed dynamically later.
- `_create_classifier`: This helper method dynamically builds the classifier part of the network.
  - It's called during the first forward pass once the input size for the linear layers is known.
- `forward`: This method defines the forward pass of the model.
  - The input `x` first passes through the `.features` layers.
  - The output from the feature extractor is flattened to determine the input size for the classifier.
  - If the `.classifier` has not been created yet, it calls `_create_classifier` to build it on the fly.
  - Finally, the flattened data is passed through the `.classifier` to produce the final output.

```

class FlexibleCNN(nn.Module):
    """
    A flexible Convolutional Neural Network with a dynamically created
    classifier.

    This CNN's architecture is defined by the provided
    hyperparameters,
    allowing for a variable number of convolutional layers. The
    classifier
    (fully connected layers) is constructed during the first forward
    pass
    to adapt to the output size of the convolutional feature
    extractor.
    """
    def __init__(self, n_layers, n_filters, kernel_sizes,
dropout_rate, fc_size):
        """
        Initializes the feature extraction part of the CNN.

        Args:
            n_layers: The number of convolutional blocks to create.
            n_filters: A list of integers specifying the number of
output
                    filters for each convolutional block.
            kernel_sizes: A list of integers specifying the kernel
size for
                    each convolutional layer.
            dropout_rate: The dropout probability to be used in the
classifier.
            fc_size: The number of neurons in the hidden fully
connected layer.
        """
        super(FlexibleCNN, self).__init__()

        # Initialize an empty list to hold the convolutional blocks
        blocks = []
        # Set the initial number of input channels for RGB images
        in_channels = 3

        # Loop to construct each convolutional block
        for i in range(n_layers):

            # Get the parameters for the current convolutional layer
            out_channels = n_filters[i]
            kernel_size = kernel_sizes[i]
            # Calculate padding to maintain the input spatial
dimensions ('same' padding)
            padding = (kernel_size - 1) // 2

            # Define a block as a sequence of Conv, ReLU, and MaxPool

```

```

layers
    block = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size,
padding=padding),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2)
    )

    # Add the newly created block to the list
    blocks.append(block)

    # Update the number of input channels for the next block
    in_channels = out_channels

    # Combine all blocks into a single feature extractor module
    self.features = nn.Sequential(*blocks)

later
    # Store hyperparameters needed for building the classifier
    self.dropout_rate = dropout_rate
    self.fc_size = fc_size

    # The classifier will be initialized dynamically in the
forward pass
    self.classifier = None

    def __create_classifier(self, flattened_size, device):
        """
        Dynamically creates and initializes the classifier part of the
network.

        This helper method is called during the first forward pass to
build the
        fully connected layers based on the feature map size from the
convolutional base.

        Args:
            flattened_size: The number of input features for the first
linear
                           layer, determined from the flattened
feature map.
            device: The device to which the new classifier layers
should be moved.
        """
        # Define the classifier's architecture
        self.classifier = nn.Sequential(
            nn.Dropout(self.dropout_rate),
            nn.Linear(flattened_size, self.fc_size),
            nn.ReLU(inplace=True),
            nn.Dropout(self.dropout_rate),

```

```

        nn.Linear(self.fc_size, 10) # Assumes 10 output classes
(e.g., CIFAR-10)
    ).to(device)

    def forward(self, x):
        """
        Defines the forward pass of the model.

        Args:
            x: The input tensor of shape (batch_size, channels,
height, width).

        Returns:
            The output logits from the classifier.
        """
        # Get the device of the input tensor to ensure consistency
        device = x.device

        # Pass the input through the feature extraction layers
        x = self.features(x)

        # Flatten the feature map to prepare it for the fully
connected layers
        flattened = torch.flatten(x, 1)
        flattened_size = flattened.size(1)

        # If the classifier has not been created yet, initialize it
        if self.classifier is None:
            self._create_classifier(flattened_size, device)

        # Pass the flattened features through the classifier to get
the final output
        return self.classifier(flattened)

```

## Defining the Optuna Objective Function

The objective function is the core of the hyperparameter optimization process, being the function that Optuna will repeatedly call to evaluate different hyperparameter configurations. This function encapsulates the entire training and evaluation process, including the definition of the CNN model architecture, the optimizer, the data loaders, the training loop, and the evaluation metrics. Within this function, you define the search space for hyperparameters using `trial.suggest_*` methods, which allow Optuna to sample hyperparameters from a defined range or set of values. For a full list of available `suggest_*` methods, you can refer to the [Optuna documentation](#).

*The objective function is designed to return a single scalar value, which represents the performance of the model for the given hyperparameters.* In your case, the aim to maximize the accuracy of the model on the validation set, which is computed using the `evaluate_accuracy` function.

**Dynamic Layer Initialization:** A noteworthy addition to this objective function is the initialization step involving a dummy input. Because the `FlexibleCNN` creates its classifier layers dynamically during the first forward pass, these parameters do not exist immediately after the model is instantiated.

- **Why use a dummy input?** Passing data through the model forces it to calculate the flattened feature size and build the classifier layers. You must do this *before* defining the optimizer so that `model.parameters()` includes the classifier weights. Otherwise, the optimizer would only track the feature extractor, leaving the classifier untrained.
- **Why these dimensions?** The tensor `torch.randn(1, 3, 32, 32)` is used to mimic the structure of the CIFAR-10 dataset. It represents a single image (batch size of 1) with 3 color channels (RGB) and a resolution of 32x32 pixels.

Observe that some hyperparameters are defined as fixed values, such as the number of epochs, the batch size, and the learning rate.

```
def objective(trial, device):  
    """  
    Defines the objective function for hyperparameter optimization  
    using Optuna.  
  
    For each trial, this function samples a set of hyperparameters,  
    constructs a model, trains it for a fixed number of epochs,  
    evaluates  
    its performance on a validation set, and returns the accuracy.  
    Optuna  
    uses the returned accuracy to guide its search for the best  
    hyperparameter combination.  
  
    Args:  
        trial: An Optuna `Trial` object, used to sample  
        hyperparameters.  
        device: The device ('cpu' or 'cuda') for model training and  
        evaluation.  
  
    Returns:  
        The validation accuracy of the trained model as a float.  
    """  
    # Sample hyperparameters for the feature extractor using the  
    # Optuna trial  
    n_layers = trial.suggest_int("n_layers", 1, 3)  
    n_filters = [trial.suggest_int(f"n_filters_{i}", 16, 128) for i in  
range(n_layers)]  
    kernel_sizes = [trial.suggest_categorical(f"kernel_size_{i}", [3,  
5]) for i in range(n_layers)]  
  
    # Sample hyperparameters for the classifier  
    dropout_rate = trial.suggest_float("dropout_rate", 0.1, 0.5)
```

```

fc_size = trial.suggest_int("fc_size", 64, 256)

# Instantiate the model with the sampled hyperparameters
model = FlexibleCNN(n_layers, n_filters, kernel_sizes,
dropout_rate, fc_size).to(device)

# Initialize the dynamic classifier layer by passing a dummy input
through the model
# This ensures all parameters are instantiated before the
optimizer is defined
dummy_input = torch.randn(1, 3, 32, 32).to(device)
model(dummy_input)

# Define fixed training parameters: learning rate, loss function,
and optimizer
learning_rate = 0.001
loss_fcn = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Define fixed data loading parameters and create data loaders
batch_size = 128
train_loader, val_loader =
helper_utils.get_dataset_dataloaders(batch_size=batch_size)

# Define the fixed number of epochs for training
n_epochs = 10
# Train the model using a helper function
helper_utils.train_model(
    model=model,
    optimizer=optimizer,
    train_dataloader=train_loader,
    n_epochs=n_epochs,
    loss_fcn=loss_fcn,
    device=device
)

# Evaluate the trained model's accuracy on the validation set
accuracy = helper_utils.evaluate_accuracy(model, val_loader,
device)

# Return the final accuracy for this trial
return accuracy

```

## Running the Optuna Study

Once that the objective function is defined, an Optuna study is created to manage the hyperparameter optimization process. The study is responsible for running the objective function multiple times with different hyperparameter configurations, allowing Optuna to explore the search space and find the best hyperparameters. In this case, your goal is to **maximize the accuracy** of the CNN model on the CIFAR-10 dataset, this is why we use

`direction='maximize'` when creating the study. The `optimize` method of the study is called to start the optimization process, which will run the objective function for a defined number of trials.

A lambda function is used to pass the device to the objective function, allowing the model to be trained on the specified device *Note:* you can also pass other parameters to the objective function using the lambda function, if needed.

**NOTE:** the code below will take about 8 minutes to run.

```
# Create a study object and optimize the objective function
study = optuna.create_study(direction='maximize') # The goal in this
case is to maximize accuracy

# Start the optimization process (it takes about 8 minutes for 20
trials)
n_trials = 20
study.optimize(lambda trial: objective(trial, device),
n_trials=n_trials) # use more trials in practice

[I 2026-01-05 09:19:10,294] A new study created in memory with name:
no-name-cf02e3e7-a6c3-45bf-a8be-d7df1c6864e1

{"model_id":"e7f7948462c746baa2bf4fa6ca553dba","version_major":2,"vers
ion_minor":0}

{"model_id":"2146bfc1f7ab49e1ae330a1d18deae29","version_major":2,"vers
ion_minor":0}

Epoch 5 - Train Loss: 1.3521
Epoch 10 - Train Loss: 1.1219
Training complete!

{"model_id":"9255a13c69da4336be930c01ef86f15b","version_major":2,"vers
ion_minor":0}

[I 2026-01-05 09:19:25,667] Trial 0 finished with value: 0.5605 and
parameters: {'n_layers': 1, 'n_filters_0': 60, 'kernel_size_0': 5,
'dropout_rate': 0.39432205269790654, 'fc_size': 74}. Best is trial 0
with value: 0.5605.

Evaluation complete.

{"model_id":"6fa62f2dca924f96aca697913eabc711","version_major":2,"vers
ion_minor":0}

{"model_id":"afe1bed3c8ab45a4b9edc93bd2c941a0","version_major":2,"vers
ion_minor":0}

Epoch 5 - Train Loss: 1.3131
Epoch 10 - Train Loss: 0.9448
```



Training complete!

```
{"model_id": "4470c95f24f64225b730b3877d727d2f", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:19:42,737] Trial 1 finished with value: 0.6015 and parameters: {'n\_layers': 3, 'n\_filters\_0': 111, 'n\_filters\_1': 110, 'n\_filters\_2': 87, 'kernel\_size\_0': 5, 'kernel\_size\_1': 3, 'kernel\_size\_2': 3, 'dropout\_rate': 0.1296556392867746, 'fc\_size': 66}. Best is trial 1 with value: 0.6015.

Evaluation complete.

```
{"model_id": "9c2c5660e0be4fe683a7d7b4a1c2d563", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2a5b6223ebe745aa8a79a74ba2761ff3", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.4107  
Epoch 10 - Train Loss: 1.1435  
Training complete!

```
{"model_id": "f5f5616698204b84b4d01bcf41ad49ef", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:19:57,218] Trial 2 finished with value: 0.564 and parameters: {'n\_layers': 3, 'n\_filters\_0': 46, 'n\_filters\_1': 33, 'n\_filters\_2': 59, 'kernel\_size\_0': 3, 'kernel\_size\_1': 5, 'kernel\_size\_2': 3, 'dropout\_rate': 0.40335918292862205, 'fc\_size': 203}. Best is trial 1 with value: 0.6015.

Evaluation complete.

```
{"model_id": "f540ab5c1957499d93d6d239fd404e2a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "95fb94042a5e4850903526763fc955f6", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1983  
Epoch 10 - Train Loss: 0.7479  
Training complete!

```
{"model_id": "d133655eb3284ea3aae96226c17485c3", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:20:13,245] Trial 3 finished with value: 0.637 and parameters: {'n\_layers': 3, 'n\_filters\_0': 83, 'n\_filters\_1': 55, 'n\_filters\_2': 112, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5,

'kernel\_size\_2': 5, 'dropout\_rate': 0.14982559796053102, 'fc\_size': 203}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "6976987a8c8f4ebe9365e85bfcec44a8", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "39a9ac63313840c19efd957c3504e5b1", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1428  
Epoch 10 - Train Loss: 0.7390  
Training complete!

```
{"model_id": "0f03230acafa49fd88c9de67a23e5b1a", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:20:27,347] Trial 4 finished with value: 0.6075 and parameters: {'n\_layers': 2, 'n\_filters\_0': 19, 'n\_filters\_1': 112, 'kernel\_size\_0': 3, 'kernel\_size\_1': 3, 'dropout\_rate': 0.17004861610732314, 'fc\_size': 199}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "b9e1631cd0b54c5e88d2ac3df40b7517", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "40d8fddd054041b7a9de3a53bf53402f", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1198  
Epoch 10 - Train Loss: 0.7321  
Training complete!

```
{"model_id": "6badc0b6afb2430a8deca080933201e2", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:20:42,165] Trial 5 finished with value: 0.577 and parameters: {'n\_layers': 1, 'n\_filters\_0': 77, 'kernel\_size\_0': 3, 'dropout\_rate': 0.16002830398310952, 'fc\_size': 127}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "731a63ca73fb4ec287c88d30830fdf64", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "beb04b5a42364a32ae2cb9c00ab3ff7b", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.4067  
Epoch 10 - Train Loss: 1.1607  
Training complete!

```
{"model_id": "8abc45aff58f435ca182bfb5d5feb826", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:20:57,643] Trial 6 finished with value: 0.574 and parameters: {'n\_layers': 2, 'n\_filters\_0': 83, 'n\_filters\_1': 45, 'kernel\_size\_0': 5, 'kernel\_size\_1': 3, 'dropout\_rate': 0.45577022982015813, 'fc\_size': 107}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "bdd8a247136b4d4c8451d035d1ca97ae", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "33f3492f02354e58ac9b1d075fec5a7f", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.5385  
Epoch 10 - Train Loss: 1.3143  
Training complete!

```
{"model_id": "785b81f553904fc5a9cf0aca94d8aac2", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:21:13,527] Trial 7 finished with value: 0.539 and parameters: {'n\_layers': 1, 'n\_filters\_0': 110, 'kernel\_size\_0': 3, 'dropout\_rate': 0.414930596327321, 'fc\_size': 96}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "fc6c8e9aa22544ecb940c9a94d420db1", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "3e36f6ff27c445f99cdcc79aba589f18", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.3984  
Epoch 10 - Train Loss: 1.1776  
Training complete!

```
{"model_id": "c709a2c3019a4f68891018abb6d524c9", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:21:29,241] Trial 8 finished with value: 0.541 and parameters: {'n\_layers': 1, 'n\_filters\_0': 98, 'kernel\_size\_0': 5,

'dropout\_rate': 0.47922837749295366, 'fc\_size': 144}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "328ae389a7ca4be9b6c6ddec7998283", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "c101f21cd0974fe684ee221569c3cf11", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1595  
Epoch 10 - Train Loss: 0.7862  
Training complete!

```
{"model_id": "19c25376f79a4bb6915c2bfb7e490936", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:21:45,031] Trial 9 finished with value: 0.62 and parameters: {'n\_layers': 2, 'n\_filters\_0': 85, 'n\_filters\_1': 55, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.24353855765290625, 'fc\_size': 187}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "6bc261e7be024be6b9236660106f4794", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "bc8faaa439584b26a88f95a75312a9b7", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1998  
Epoch 10 - Train Loss: 0.7118  
Training complete!

```
{"model_id": "10bebca5611e45909f3833f3ea46e2e9", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:21:59,623] Trial 10 finished with value: 0.6005 and parameters: {'n\_layers': 3, 'n\_filters\_0': 33, 'n\_filters\_1': 80, 'n\_filters\_2': 128, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'kernel\_size\_2': 5, 'dropout\_rate': 0.28534361487683313, 'fc\_size': 246}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "0c935f2a1d0245349b214a7e74b61e28", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e03e39b4f9d04790a5a9edb0bbb6c377", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.2027  
Epoch 10 - Train Loss: 0.8235  
Training complete!

```
{"model_id": "576b6e52203b46e997bd0de470161ce0", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:22:16,946] Trial 11 finished with value: 0.5985 and parameters: {'n\_layers': 2, 'n\_filters\_0': 128, 'n\_filters\_1': 65, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.2415725562749732, 'fc\_size': 181}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "a7607b7ffc3b4b73b7c4c3fa45f4d2dd", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b12cf1f6bdae42e6ae7f5a9d80d3d111", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1213  
Epoch 10 - Train Loss: 0.6537  
Training complete!

```
{"model_id": "48005f90e3bd413c96a9f7ef6b27b929", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:22:32,014] Trial 12 finished with value: 0.6075 and parameters: {'n\_layers': 2, 'n\_filters\_0': 66, 'n\_filters\_1': 62, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.22862864982465042, 'fc\_size': 240}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "2814b11c922b4d5fba66aa78e42c7393", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "73d6d8ca6fad4da29414cedfc8e78c6d", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.2792  
Epoch 10 - Train Loss: 0.9074  
Training complete!

```
{"model_id": "055658948bd54b178509426cdbabe5b6", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:22:47,962] Trial 13 finished with value: 0.6035 and parameters: {'n\_layers': 3, 'n\_filters\_0': 87, 'n\_filters\_1': 23, 'n\_filters\_2': 122, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'kernel\_size\_2': 5, 'dropout\_rate': 0.3267847585021573, 'fc\_size': 169}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "b603b2c402094b2dbdd2db0a170ab98a", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "56c67087790642bbb297a17e812ae40e", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.0873  
Epoch 10 - Train Loss: 0.6149  
Training complete!

```
{"model_id": "212a23c2055445638835e205f4ad129d", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:23:03,281] Trial 14 finished with value: 0.611 and parameters: {'n\_layers': 2, 'n\_filters\_0': 57, 'n\_filters\_1': 87, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.21395737744497967, 'fc\_size': 216}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "5235049d55df4a47b1e46e5f2d559ba5", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "2fb7516b114a4e139c0d937cb6161786", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.3255  
Epoch 10 - Train Loss: 0.9646  
Training complete!

```
{"model_id": "dbf2c8f7aaf24f05b292844853bd9942", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:23:19,690] Trial 15 finished with value: 0.5675 and parameters: {'n\_layers': 3, 'n\_filters\_0': 97, 'n\_filters\_1': 48, 'n\_filters\_2': 23, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'kernel\_size\_2': 5, 'dropout\_rate': 0.10598056721487074, 'fc\_size': 154}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "0670b4bfe0d64cbb9d5bb15c040d7ff7", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "01c58583165c4745b9fe3b7fb9c1fb9d", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.1978  
Epoch 10 - Train Loss: 0.8292  
Training complete!

```
{"model_id": "4b80039172e34bc29ab85b0b650a0b45", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:23:34,921] Trial 16 finished with value: 0.595 and parameters: {'n\_layers': 2, 'n\_filters\_0': 74, 'n\_filters\_1': 47, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.3065312070720916, 'fc\_size': 224}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "6158d6a1ba4946219e722099dbf6dc21", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "4a5ab078e7f841a2a76cda9a708e4b78", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 1.0683  
Epoch 10 - Train Loss: 0.5940  
Training complete!

```
{"model_id": "96bbfb037b364b17a14514bdc7bef5bb", "version_major": 2, "version_minor": 0}
```

[I 2026-01-05 09:23:51,830] Trial 17 finished with value: 0.627 and parameters: {'n\_layers': 2, 'n\_filters\_0': 100, 'n\_filters\_1': 87, 'kernel\_size\_0': 5, 'kernel\_size\_1': 5, 'dropout\_rate': 0.18841358224443872, 'fc\_size': 179}. Best is trial 3 with value: 0.637.

Evaluation complete.

```
{"model_id": "05447240a90e422d9362c53c5d88c93f", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "e8dec68ee2a6488bb3a326d6ad425539", "version_major": 2, "version_minor": 0}
```

```
Epoch 5 - Train Loss: 1.2202
Epoch 10 - Train Loss: 0.7537
Training complete!
```

```
{"model_id": "71af9d95689d4533abfef3873af53369", "version_major": 2, "version_minor": 0}
```

```
[I 2026-01-05 09:24:09,213] Trial 18 finished with value: 0.639 and
parameters: {'n_layers': 3, 'n_filters_0': 123, 'n_filters_1': 93,
'n_filters_2': 92, 'kernel_size_0': 3, 'kernel_size_1': 3,
'kernel_size_2': 5, 'dropout_rate': 0.17970444647999234, 'fc_size':
173}. Best is trial 18 with value: 0.639.
```

```
Evaluation complete.
```

```
{"model_id": "d913d62ead7e4df487d84ba9d7ed96ae", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "77bcff4d21d74fc8aae21bb243dbb699", "version_major": 2, "version_minor": 0}
```

```
Epoch 5 - Train Loss: 1.1572
Epoch 10 - Train Loss: 0.6740
Training complete!
```

```
{"model_id": "8cae6b4a20d641f3bd46de6fa7bfbce9", "version_major": 2, "version_minor": 0}
```

```
[I 2026-01-05 09:24:26,882] Trial 19 finished with value: 0.6215 and
parameters: {'n_layers': 3, 'n_filters_0': 126, 'n_filters_1': 98,
'n_filters_2': 94, 'kernel_size_0': 3, 'kernel_size_1': 3,
'kernel_size_2': 5, 'dropout_rate': 0.10026737302441634, 'fc_size':
139}. Best is trial 18 with value: 0.639.
```

```
Evaluation complete.
```

## Analyzing the Results

After the optimization process is complete, you can analyze the results to understand which hyperparameters yielded the best performance. The `study` object contains a wealth of information about the trials, including the hyperparameters sampled, the corresponding performance metrics, and the best trial.

You can access the full DataFrame of trials using the `trials_dataframe()` method, which provides a comprehensive overview of all the trials conducted during the optimization process. This DataFrame includes columns for the trial number, hyperparameters, and the objective value (in our case, the accuracy).

To access the best hyperparameters and the best trial, you can use the `best_trial` attributes of the study object.



**Note:** these results may change every time you re-run the training study.

```
# Extract the dataframe with the results
```

```
df = study.trials_dataframe()
```

```
df
```

|                     | number | value  |                            | datetime_start |  |
|---------------------|--------|--------|----------------------------|----------------|--|
| datetime_complete \ |        |        |                            |                |  |
| 0                   | 0      | 0.5605 | 2026-01-05 09:19:10.296147 | 2026-01-05     |  |
| 09:19:25.666770     |        |        |                            |                |  |
| 1                   | 1      | 0.6015 | 2026-01-05 09:19:25.667709 | 2026-01-05     |  |
| 09:19:42.737588     |        |        |                            |                |  |
| 2                   | 2      | 0.5640 | 2026-01-05 09:19:42.738584 | 2026-01-05     |  |
| 09:19:57.218522     |        |        |                            |                |  |
| 3                   | 3      | 0.6370 | 2026-01-05 09:19:57.219525 | 2026-01-05     |  |
| 09:20:13.245503     |        |        |                            |                |  |
| 4                   | 4      | 0.6075 | 2026-01-05 09:20:13.246347 | 2026-01-05     |  |
| 09:20:27.347648     |        |        |                            |                |  |
| 5                   | 5      | 0.5770 | 2026-01-05 09:20:27.348546 | 2026-01-05     |  |
| 09:20:42.165136     |        |        |                            |                |  |
| 6                   | 6      | 0.5740 | 2026-01-05 09:20:42.165760 | 2026-01-05     |  |
| 09:20:57.643155     |        |        |                            |                |  |
| 7                   | 7      | 0.5390 | 2026-01-05 09:20:57.644110 | 2026-01-05     |  |
| 09:21:13.527309     |        |        |                            |                |  |
| 8                   | 8      | 0.5410 | 2026-01-05 09:21:13.528074 | 2026-01-05     |  |
| 09:21:29.241386     |        |        |                            |                |  |
| 9                   | 9      | 0.6200 | 2026-01-05 09:21:29.242279 | 2026-01-05     |  |
| 09:21:45.031427     |        |        |                            |                |  |
| 10                  | 10     | 0.6005 | 2026-01-05 09:21:45.032212 | 2026-01-05     |  |
| 09:21:59.623041     |        |        |                            |                |  |
| 11                  | 11     | 0.5985 | 2026-01-05 09:21:59.624094 | 2026-01-05     |  |
| 09:22:16.946014     |        |        |                            |                |  |
| 12                  | 12     | 0.6075 | 2026-01-05 09:22:16.946665 | 2026-01-05     |  |
| 09:22:32.014473     |        |        |                            |                |  |
| 13                  | 13     | 0.6035 | 2026-01-05 09:22:32.015209 | 2026-01-05     |  |
| 09:22:47.962430     |        |        |                            |                |  |
| 14                  | 14     | 0.6110 | 2026-01-05 09:22:47.963397 | 2026-01-05     |  |
| 09:23:03.281613     |        |        |                            |                |  |
| 15                  | 15     | 0.5675 | 2026-01-05 09:23:03.282431 | 2026-01-05     |  |
| 09:23:19.689889     |        |        |                            |                |  |
| 16                  | 16     | 0.5950 | 2026-01-05 09:23:19.690540 | 2026-01-05     |  |
| 09:23:34.921565     |        |        |                            |                |  |
| 17                  | 17     | 0.6270 | 2026-01-05 09:23:34.922289 | 2026-01-05     |  |
| 09:23:51.830098     |        |        |                            |                |  |
| 18                  | 18     | 0.6390 | 2026-01-05 09:23:51.831069 | 2026-01-05     |  |
| 09:24:09.213669     |        |        |                            |                |  |
| 19                  | 19     | 0.6215 | 2026-01-05 09:24:09.214319 | 2026-01-05     |  |
| 09:24:26.882125     |        |        |                            |                |  |

|    |        | duration        | params_dropout_rate | params_fc_size | \ |
|----|--------|-----------------|---------------------|----------------|---|
| 0  | 0 days | 00:00:15.370623 | 0.394322            | 74             |   |
| 1  | 0 days | 00:00:17.069879 | 0.129656            | 66             |   |
| 2  | 0 days | 00:00:14.479938 | 0.403359            | 203            |   |
| 3  | 0 days | 00:00:16.025978 | 0.149826            | 203            |   |
| 4  | 0 days | 00:00:14.101301 | 0.170049            | 199            |   |
| 5  | 0 days | 00:00:14.816590 | 0.160028            | 127            |   |
| 6  | 0 days | 00:00:15.477395 | 0.455770            | 107            |   |
| 7  | 0 days | 00:00:15.883199 | 0.414931            | 96             |   |
| 8  | 0 days | 00:00:15.713312 | 0.479228            | 144            |   |
| 9  | 0 days | 00:00:15.789148 | 0.243539            | 187            |   |
| 10 | 0 days | 00:00:14.590829 | 0.285344            | 246            |   |
| 11 | 0 days | 00:00:17.321920 | 0.241573            | 181            |   |
| 12 | 0 days | 00:00:15.067808 | 0.228629            | 240            |   |
| 13 | 0 days | 00:00:15.947221 | 0.326785            | 169            |   |
| 14 | 0 days | 00:00:15.318216 | 0.213957            | 216            |   |
| 15 | 0 days | 00:00:16.407458 | 0.105981            | 154            |   |
| 16 | 0 days | 00:00:15.231025 | 0.306531            | 224            |   |
| 17 | 0 days | 00:00:16.907809 | 0.188414            | 179            |   |
| 18 | 0 days | 00:00:17.382600 | 0.179704            | 173            |   |
| 19 | 0 days | 00:00:17.667806 | 0.100267            | 139            |   |

|                      | params_kernel_size_0 | params_kernel_size_1 |     |
|----------------------|----------------------|----------------------|-----|
| params_kernel_size_2 | \                    |                      |     |
| 0                    | 5                    | NaN                  | NaN |
| 1                    | 5                    | 3.0                  | 3.0 |
| 2                    | 3                    | 5.0                  | 3.0 |
| 3                    | 5                    | 5.0                  | 5.0 |
| 4                    | 3                    | 3.0                  | NaN |
| 5                    | 3                    | NaN                  | NaN |
| 6                    | 5                    | 3.0                  | NaN |
| 7                    | 3                    | NaN                  | NaN |
| 8                    | 5                    | NaN                  | NaN |
| 9                    | 5                    | 5.0                  | NaN |
| 10                   | 5                    | 5.0                  | 5.0 |
| 11                   | 5                    | 5.0                  | NaN |
| 12                   | 5                    | 5.0                  | NaN |
| 13                   | 5                    | 5.0                  | 5.0 |

|    |   |     |     |
|----|---|-----|-----|
| 14 | 5 | 5.0 | NaN |
| 15 | 5 | 5.0 | 5.0 |
| 16 | 5 | 5.0 | NaN |
| 17 | 5 | 5.0 | NaN |
| 18 | 3 | 3.0 | 5.0 |
| 19 | 3 | 3.0 | 5.0 |

|    | params_n_filters_0 | params_n_filters_1 | params_n_filters_2 \ |
|----|--------------------|--------------------|----------------------|
| 0  | 60                 | NaN                | NaN                  |
| 1  | 111                | 110.0              | 87.0                 |
| 2  | 46                 | 33.0               | 59.0                 |
| 3  | 83                 | 55.0               | 112.0                |
| 4  | 19                 | 112.0              | NaN                  |
| 5  | 77                 | NaN                | NaN                  |
| 6  | 83                 | 45.0               | NaN                  |
| 7  | 110                | NaN                | NaN                  |
| 8  | 98                 | NaN                | NaN                  |
| 9  | 85                 | 55.0               | NaN                  |
| 10 | 33                 | 80.0               | 128.0                |
| 11 | 128                | 65.0               | NaN                  |
| 12 | 66                 | 62.0               | NaN                  |
| 13 | 87                 | 23.0               | 122.0                |
| 14 | 57                 | 87.0               | NaN                  |
| 15 | 97                 | 48.0               | 23.0                 |
| 16 | 74                 | 47.0               | NaN                  |
| 17 | 100                | 87.0               | NaN                  |
| 18 | 123                | 93.0               | 92.0                 |
| 19 | 126                | 98.0               | 94.0                 |

|    | params_n_layers | state    |
|----|-----------------|----------|
| 0  | 1               | COMPLETE |
| 1  | 3               | COMPLETE |
| 2  | 3               | COMPLETE |
| 3  | 3               | COMPLETE |
| 4  | 2               | COMPLETE |
| 5  | 1               | COMPLETE |
| 6  | 2               | COMPLETE |
| 7  | 1               | COMPLETE |
| 8  | 1               | COMPLETE |
| 9  | 2               | COMPLETE |
| 10 | 3               | COMPLETE |
| 11 | 2               | COMPLETE |
| 12 | 2               | COMPLETE |

```

13             3 COMPLETE
14             2 COMPLETE
15             3 COMPLETE
16             2 COMPLETE
17             2 COMPLETE
18             3 COMPLETE
19             3 COMPLETE

# Extract and print the best trial
best_trial = study.best_trial

print("Best trial:")
print(f"  Value (Accuracy): {best_trial.value:.4f}")

print("  Hyperparameters:")
pprint(best_trial.params)

Best trial:
  Value (Accuracy): 0.6390
  Hyperparameters:
{'dropout_rate': 0.17970444647999234,
 'fc_size': 173,
 'kernel_size_0': 3,
 'kernel_size_1': 3,
 'kernel_size_2': 5,
 'n_filters_0': 123,
 'n_filters_1': 93,
 'n_filters_2': 92,
 'n_layers': 3}

```

## Visualizing the Results

Optuna provides several built-in visualization functions to help analyze the results of the hyperparameter optimization process. These visualizations can provide valuable insights into the optimization process and the impact of different hyperparameters on the model's performance:

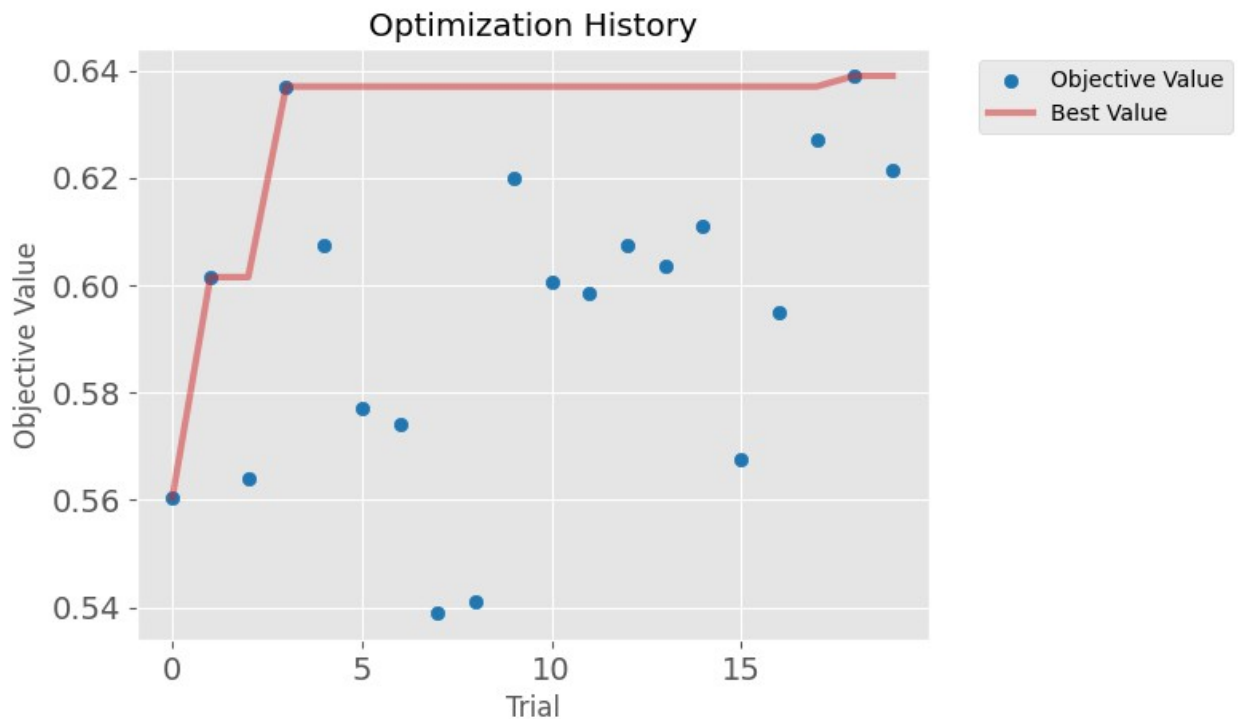
- **plot\_optimization\_history**: This plot shows the optimization history of the objective function, allowing you to see how the performance of the model improved over time. It provides a visual representation of the objective values (in this case, accuracy) across different trials.
- **plot\_param\_importances**: This plot shows the importance of each hyperparameter in the optimization process. It helps identify which hyperparameters had the most significant impact on the model's performance, allowing you to focus on the most influential hyperparameters in future experiments.

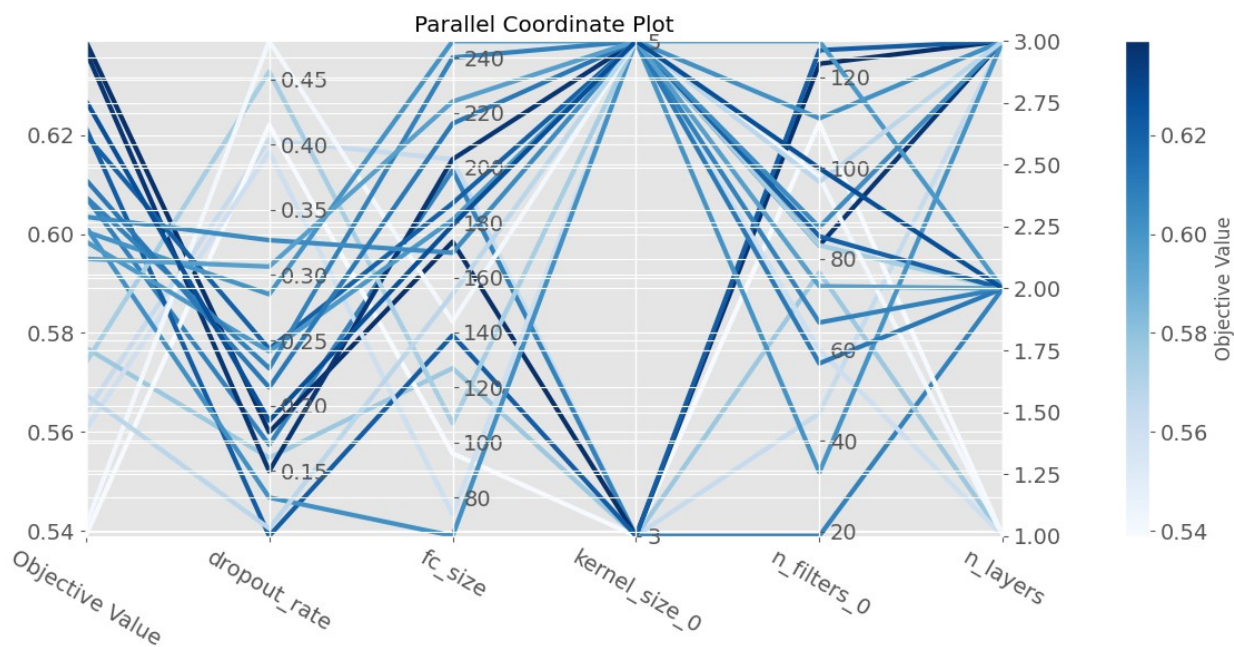
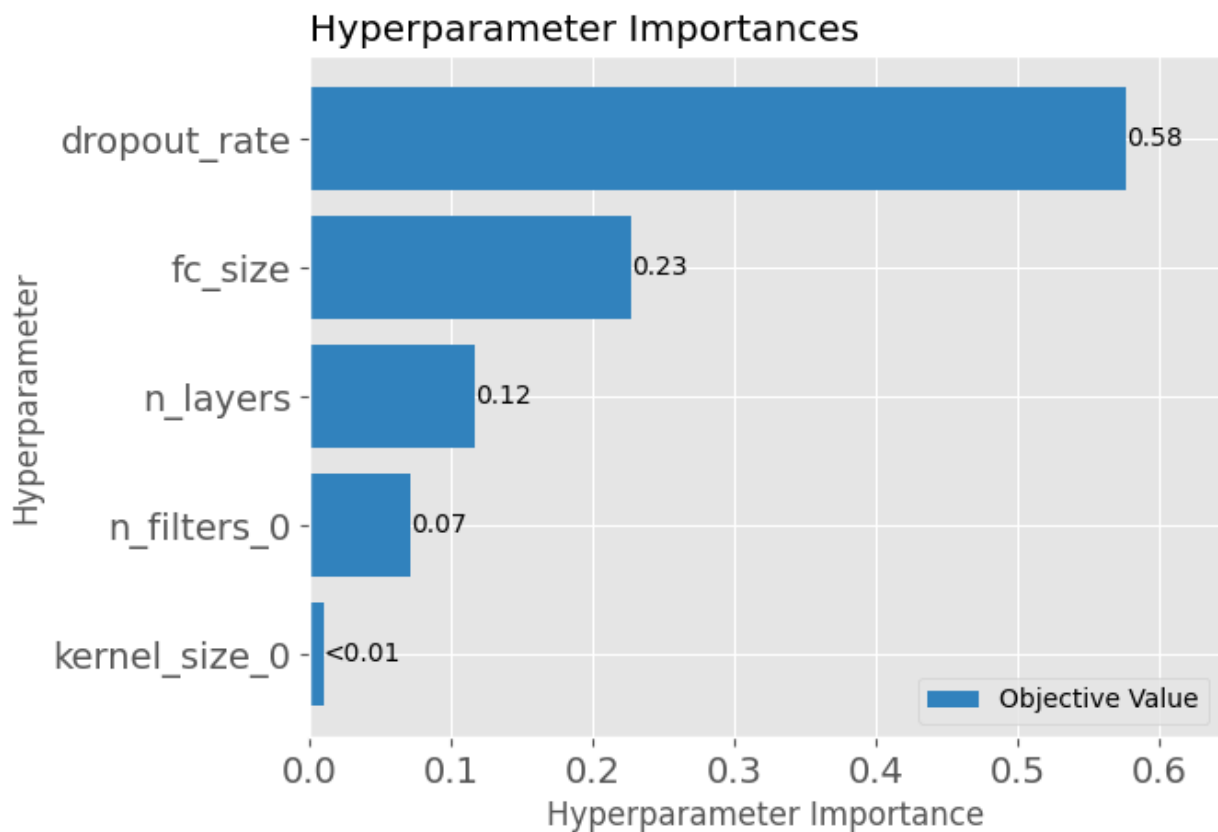
- `plot_parallel_coordinate`: This plot visualizes the relationship between different hyperparameters and the objective function. It allows you to see how different hyperparameter configurations affected the model's performance, providing insights into the interactions between hyperparameters and their impact on the objective value.

```
# Plotting the optimization history
optuna.visualization.matplotlib.plot_optimization_history(study)
plt.title('Optimization History')
plt.show()

# Importance of hyperparameters
optuna.visualization.matplotlib.plot_param_importances(study)
plt.show()

ax = optuna.visualization.matplotlib.plot_parallel_coordinate(
    study, params=['n_layers', 'n_filters_0', 'kernel_size_0',
                  'dropout_rate', 'fc_size']
)
fig = ax.figure
fig.set_size_inches(12, 6, forward=True) # forward=True updates the canvas
fig.tight_layout()
```





## (Optional) Comparing Grid Search with Optuna's Default Search

In this section, you will compare the performance of Optuna's default search algorithm with a grid search approach. Grid search is a traditional method for hyperparameter optimization that exhaustively searches through a predefined set of hyperparameter combinations. While it can be effective, it is often computationally expensive and may not explore the search space as efficiently as Optuna's adaptive algorithms.

To perform the comparison, a simpler example is used, such as the one from previous labs. The `FlexibleSimpleCNN` model is used on a smaller dataset, consisting in a subset of the [Fruit and Vegetable Disease \(Healthy vs Rotten\)](#). This dataset contains images of healthy and rotten fruits and vegetables. You will only use a custom subset of the dataset, which contains 1200 images of apples, 1000 being healthy and 200 being rotten.

```
class FlexibleSimpleCNN(nn.Module):
    """
    A simple, flexible Convolutional Neural Network.

    This network consists of two convolutional layers, each followed
    by a
    max-pooling layer, and two fully connected layers. The number of
    filters
    in the convolutional layers and the size of the hidden linear
    layer are
    configurable, making the architecture adaptable to different
    requirements.
    """
    def __init__(self, conv1_out, conv2_out, fc_size, num_classes):
        """
        Initializes the layers of the CNN.

        Args:
            conv1_out: The number of output channels for the first
                       convolutional layer.
            conv2_out: The number of output channels for the second
                       convolutional layer.
            fc_size: The number of neurons in the hidden fully
connected layer.
            num_classes: The number of output classes for the final
layer.
        """
        super(FlexibleSimpleCNN, self).__init__()
        # Define the first convolutional layer
        self.conv1 = nn.Conv2d(3, conv1_out, kernel_size=3, padding=1)
        # Define the second convolutional layer
        self.conv2 = nn.Conv2d(conv1_out, conv2_out, kernel_size=3,
padding=1)
        # Define a max pooling layer to be used after each convolution
```

```

        self.pool = nn.MaxPool2d(2, 2)

        # Define the first fully connected (hidden) layer
        # Assumes input images are 32x32, resulting in an 8x8 feature
        map after two pooling layers
        self.fc1 = nn.Linear(conv2_out * 8 * 8, fc_size)
        # Define the final fully connected (output) layer
        self.fc2 = nn.Linear(fc_size, num_classes)

    def forward(self, x):
        """
        Defines the forward pass of the network.

        Args:
            x: The input tensor of shape (batch_size, channels,
            height, width).

        Returns:
            The output logits from the network.
        """
        # Apply the first convolutional block: convolution, ReLU
        activation, and pooling
        x = self.pool(F.relu(self.conv1(x)))
        # Apply the second convolutional block
        x = self.pool(F.relu(self.conv2(x)))
        # Flatten the feature map to prepare for the fully connected
        layers
        x = x.view(x.size(0), -1)
        # Pass through the first fully connected layer with ReLU
        activation
        x = F.relu(self.fc1(x))
        # Pass through the final output layer
        x = self.fc2(x)
        # Return the resulting logits
        return x

```

The goal of the model is to classify the images into two classes: healthy and rotten. Note that the dataset is imbalanced, with a higher number of healthy images compared to rotten ones. In such cases, accuracy is not the best metric to evaluate model performance, as it can be misleading due to the class imbalance. Therefore, the **F1 score will be used as the evaluation metric**, as it is more suitable for imbalanced datasets. To ensure a fair comparison between grid search and Optuna's search, the same hyperparameter space will be used for both approaches.

You will also use the `trial.set_user_attr` method, which allows you to store additional information in each trial object, such as accuracy, recall, precision, and F1 score. This is useful for tracking additional metrics that can be analyzed later.

**Why no dummy input is needed here:** Unlike the `FlexibleCNN` used earlier, the `FlexibleSimpleCNN` used in this section does not require a dummy input to initialize its parameters.



- **Static Initialization:** In `FlexibleSimpleCNN`, the classifier layers (`self.fc1` and `self.fc2`) are defined directly within the `__init__` method based on a fixed input, rather than being created dynamically during the first forward pass.
- **Optimizer Safety:** Because these layers exist immediately upon model instantiation, `model.parameters()` correctly includes the classifier weights right from the start. The optimizer will therefore track and train the classifier correctly without needing a "warm-up" pass.

```
def objective_apples(trial, device):
    """
    Defines the Optuna objective function for a CNN on an apple
    dataset.

    For each trial, this function samples hyperparameters for a CNN
    architecture, trains the model on a custom apple dataset, and
    evaluates
    its performance. It logs accuracy, precision, and recall, while
    returning the F1-score as the primary metric for Optuna to
    optimize.

    Args:
        trial: An Optuna `Trial` object used to sample
        hyperparameters.
        device: The device ('cpu' or 'cuda') for model training and
        evaluation.

    Returns:
        The F1-score of the trained model on the validation set.
    """
    # Sample a set of hyperparameters for the model architecture
    conv1_out = trial.suggest_int("conv1_out", 8, 64, step=8)
    conv2_out = trial.suggest_int("conv2_out", 16, 128, step=16)
    fc_size = trial.suggest_int("fc_size", 32, 256, step=32)

    # Define fixed parameters for the data loaders
    img_size = 32
    batch_size = 128

    # Create the training and validation data loaders for the apple
    dataset
    train_loader, val_loader =
    helper_utils.get_apples_dataset_data loaders(
        img_size=img_size,
        batch_size=batch_size
    )

    # Specify the number of output classes for the dataset
    num_classes = 2
    # Create an instance of the model with the sampled hyperparameters
```

```

model = FlexibleSimpleCNN(
    conv1_out=conv1_out,
    conv2_out=conv2_out,
    fc_size=fc_size,
    num_classes=num_classes
).to(device)

# Define fixed training components: learning rate, optimizer, and
loss function
learning_rate = 0.001
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
loss_fcn = nn.CrossEntropyLoss()

# Set the fixed number of epochs for the training loop
n_epochs = 5
# Train the model using a helper function
helper_utils.train_model(
    model=model,
    optimizer=optimizer,
    train_dataloader=train_loader,
    n_epochs=n_epochs,
    loss_fcn=loss_fcn,
    device=device
)

# Evaluate the trained model on the validation set to get
performance metrics
accuracy, precision, recall, f1 = helper_utils.evaluate_metrics(
    model, val_loader, device, num_classes=2
)

# Log additional metrics to the Optuna trial for more detailed
analysis
trial.set_user_attr("accuracy", accuracy)
trial.set_user_attr("precision", precision)
trial.set_user_attr("recall", recall)

# Return the F1-score as the objective value for Optuna to
maximize
return f1

```

With the objective function defined, you can now run both the default Optuna search algorithm and the grid search algorithm. Both methods will be executed for the same number of trials, and the results will be compared using the `plot_optimization_history` function.

First, in this case, you will specify `sampler = optuna.samplers.TPESampler(seed=42)` (the default sampler in Optuna), which is the Tree-structured Parzen Estimator (TPE) sampler. The TPE sampler is a sophisticated algorithm that adapts the search space based on the results of previous trials, allowing for more efficient exploration of the hyperparameter space. Then, you will create a study object with the desired sampler and set `direction='maximize'` to

indicate that you want to maximize the objective function (in this case, the F1 score). Note that the trials dataframe now includes additional columns for accuracy, recall, and precision, which were set using the `trial.set_user_attr` method inside the objective function.

Next, run the grid search sampler (`sampler = optuna.samplers.GridSampler(param_grid)`) to explore the hyperparameter space. The grid search will iterate, until reaching the number of trials defined, over all possible combinations of hyperparameters in the grid, evaluating the objective function for each unique configuration without repeating any combination.

```
seed = 42
helper_utils.set_seed(seed)

sampler = optuna.samplers.TPESampler(seed=seed)  # Use TPE sampler
                                                (the default sampler in Optuna)

# Create a study object and optimize the objective function
study_apples = optuna.create_study(direction='maximize',
sampler=sampler)

n_trials = 10
study_apples.optimize(lambda trial: objective_apples(trial, device),
n_trials=n_trials)

[I 2026-01-05 09:41:21,116] A new study created in memory with name:
no-name-a73a6a91-3fcd-465b-8089-81d096e1412c

{'healthy': 0, 'rotten': 1}

{"model_id": "d5b29926f2b3497baa090c75f514653f", "version_major": 2, "version_minor": 0}

{"model_id": "d410a53e4720499cb8ab8cbfd32d3394", "version_major": 2, "version_minor": 0}

Epoch 5 - Train Loss: 0.3548
Training complete!

[I 2026-01-05 09:42:10,972] Trial 0 finished with value:
0.6255446672439575 and parameters: {'conv1_out': 24, 'conv2_out': 128,
'fc_size': 192}. Best is trial 0 with value: 0.6255446672439575.

{'healthy': 0, 'rotten': 1}

{"model_id": "9b5f270551494e9489ad82cc8a6c6f5c", "version_major": 2, "version_minor": 0}

{"model_id": "ed82e8b97dd347688046dd9a53471551", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 0.4515  
Training complete!

[I 2026-01-05 09:42:55,116] Trial 1 finished with value:  
0.439461886882782 and parameters: {'conv1\_out': 40, 'conv2\_out': 32,  
'fc\_size': 64}. Best is trial 0 with value: 0.6255446672439575.

{'healthy': 0, 'rotten': 1}

{"model\_id": "00f4d8163b7d485792b2ffeeee33a2d9", "version\_major": 2, "version\_minor": 0}

{"model\_id": "0a1511bbdc8c497dbf1c85bd7dfdb934", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3359  
Training complete!

[I 2026-01-05 09:43:37,400] Trial 2 finished with value:  
0.7176079750061035 and parameters: {'conv1\_out': 8, 'conv2\_out': 112,  
'fc\_size': 160}. Best is trial 2 with value: 0.7176079750061035.

{'healthy': 0, 'rotten': 1}

{"model\_id": "cdd16eb261ee471fb14d6c358f6cb1db", "version\_major": 2, "version\_minor": 0}

{"model\_id": "8a13cb46681c417884a5a836f6a4315d", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3462  
Training complete!

[I 2026-01-05 09:44:17,947] Trial 3 finished with value:  
0.6919227838516235 and parameters: {'conv1\_out': 48, 'conv2\_out': 16,  
'fc\_size': 256}. Best is trial 2 with value: 0.7176079750061035.

{'healthy': 0, 'rotten': 1}

{"model\_id": "744c0ca2a0c94a128724e6254124a9ed", "version\_major": 2, "version\_minor": 0}

{"model\_id": "41b20f66bdc64fc4bcd2674529dc1313", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3534  
Training complete!

[I 2026-01-05 09:44:59,895] Trial 4 finished with value: 0.8412935733795166 and parameters: {'conv1\_out': 56, 'conv2\_out': 32, 'fc\_size': 64}. Best is trial 4 with value: 0.8412935733795166.

{'healthy': 0, 'rotten': 1}

{"model\_id": "90762d5c86ef4a3f998679f3911a1b0c", "version\_major": 2, "version\_minor": 0}

{"model\_id": "a8912ba8bac3495094e364746911bb2e", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3716  
Training complete!

[I 2026-01-05 09:45:41,655] Trial 5 finished with value: 0.5040310621261597 and parameters: {'conv1\_out': 16, 'conv2\_out': 48, 'fc\_size': 160}. Best is trial 4 with value: 0.8412935733795166.

{'healthy': 0, 'rotten': 1}

{"model\_id": "52dca1b0ef264e16a900fd650ede1649", "version\_major": 2, "version\_minor": 0}

{"model\_id": "0b6b89c1cec1486b9317ec7f95b2cd81", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.2958  
Training complete!

[I 2026-01-05 09:46:23,515] Trial 6 finished with value: 0.7342193126678467 and parameters: {'conv1\_out': 32, 'conv2\_out': 48, 'fc\_size': 160}. Best is trial 4 with value: 0.8412935733795166.

{'healthy': 0, 'rotten': 1}

{"model\_id": "66747e706e964f52932c99e4de721458", "version\_major": 2, "version\_minor": 0}

{"model\_id": "e794e86cf9f2443b9f39ec9dc82d0b81", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.2837  
Training complete!

[I 2026-01-05 09:47:04,248] Trial 7 finished with value: 0.858942985534668 and parameters: {'conv1\_out': 16, 'conv2\_out': 48, 'fc\_size': 96}. Best is trial 7 with value: 0.858942985534668.

{'healthy': 0, 'rotten': 1}

```
{"model_id":"ea106758f1d7499a823a46729f1e6c52","version_major":2,"version_minor":0}
```

```
{"model_id":"09ff44ddde6f4f73a6c6b7042b0680a4","version_major":2,"version_minor":0}
```

Epoch 5 - Train Loss: 0.3144  
Training complete!

[I 2026-01-05 09:47:46,058] Trial 8 finished with value: 0.9125000238418579 and parameters: {'conv1\_out': 32, 'conv2\_out': 112, 'fc\_size': 64}. Best is trial 8 with value: 0.9125000238418579.

```
{'healthy': 0, 'rotten': 1}
```

```
{"model_id":"1b0d413ab1b346eb86e0b00f55ca5268","version_major":2,"version_minor":0}
```

```
{"model_id":"cf7a8838926d442996ba43f5f6e3547b","version_major":2,"version_minor":0}
```

Epoch 5 - Train Loss: 0.3693  
Training complete!

[I 2026-01-05 09:48:27,236] Trial 9 finished with value: 0.6905393600463867 and parameters: {'conv1\_out': 40, 'conv2\_out': 80, 'fc\_size': 32}. Best is trial 8 with value: 0.9125000238418579.

```
df_apples_study = study_apples.trials_dataframe()
```

```
df_apples_study
```

|                     | number | value    |                            | datetime_start             |  |
|---------------------|--------|----------|----------------------------|----------------------------|--|
| datetime_complete \ |        |          |                            |                            |  |
| 0                   | 0      | 0.625545 | 2026-01-05 09:41:21.117513 | 2026-01-05 09:42:10.972413 |  |
| 1                   | 1      | 0.439462 | 2026-01-05 09:42:10.973190 | 2026-01-05 09:42:55.116708 |  |
| 2                   | 2      | 0.717608 | 2026-01-05 09:42:55.117453 | 2026-01-05 09:43:37.399890 |  |
| 3                   | 3      | 0.691923 | 2026-01-05 09:43:37.400650 | 2026-01-05 09:44:17.947388 |  |
| 4                   | 4      | 0.841294 | 2026-01-05 09:44:17.948175 | 2026-01-05 09:44:59.895205 |  |
| 5                   | 5      | 0.504031 | 2026-01-05 09:44:59.896329 | 2026-01-05 09:45:41.654855 |  |
| 6                   | 6      | 0.734219 | 2026-01-05 09:45:41.655636 | 2026-01-05 09:46:23.515570 |  |
| 7                   | 7      | 0.858943 | 2026-01-05 09:46:23.516313 | 2026-01-05 09:47:04.248807 |  |

```

8      8  0.912500 2026-01-05 09:47:04.249543 2026-01-05
09:47:46.058164
9      9  0.690539 2026-01-05 09:47:46.058897 2026-01-05
09:48:27.236790

```

|                          | duration | params_conv1_out | params_conv2_out |
|--------------------------|----------|------------------|------------------|
| params_fc_size \         |          |                  |                  |
| 0 0 days 00:00:49.854900 | 24       | 128              |                  |
| 192                      |          |                  |                  |
| 1 0 days 00:00:44.143518 | 40       | 32               |                  |
| 64                       |          |                  |                  |
| 2 0 days 00:00:42.282437 | 8        | 112              |                  |
| 160                      |          |                  |                  |
| 3 0 days 00:00:40.546738 | 48       | 16               |                  |
| 256                      |          |                  |                  |
| 4 0 days 00:00:41.947030 | 56       | 32               |                  |
| 64                       |          |                  |                  |
| 5 0 days 00:00:41.758526 | 16       | 48               |                  |
| 160                      |          |                  |                  |
| 6 0 days 00:00:41.859934 | 32       | 48               |                  |
| 160                      |          |                  |                  |
| 7 0 days 00:00:40.732494 | 16       | 48               |                  |
| 96                       |          |                  |                  |
| 8 0 days 00:00:41.808621 | 32       | 112              |                  |
| 64                       |          |                  |                  |
| 9 0 days 00:00:41.177893 | 40       | 80               |                  |
| 32                       |          |                  |                  |

|          | user_attrs_accuracy | user_attrs_precision | user_attrs_recall |
|----------|---------------------|----------------------|-------------------|
| state    |                     |                      |                   |
| 0        | 0.824               | 0.839588             | 0.606009          |
| COMPLETE |                     |                      |                   |
| 1        | 0.784               | 0.392000             | 0.500000          |
| COMPLETE |                     |                      |                   |
| 2        | 0.864               | 0.860886             | 0.676211          |
| COMPLETE |                     |                      |                   |
| 3        | 0.868               | 0.807815             | 0.654647          |
| COMPLETE |                     |                      |                   |
| 4        | 0.932               | 0.925148             | 0.792602          |
| COMPLETE |                     |                      |                   |
| 5        | 0.812               | 0.904858             | 0.530000          |
| COMPLETE |                     |                      |                   |
| 6        | 0.872               | 0.930736             | 0.686275          |
| COMPLETE |                     |                      |                   |
| 7        | 0.916               | 0.899405             | 0.830574          |
| COMPLETE |                     |                      |                   |
| 8        | 0.944               | 0.912500             | 0.912500          |
| COMPLETE |                     |                      |                   |

```
9          0.860          0.847701          0.652185
COMPLETE
```

```
# Run with Grid Search Sampler
```

```
# Define the hyperparameter grid
```

```
param_grid = {
    "conv1_out": list(range(8, 65, 8)),      # [8, 16, 24, 32, 40,
48, 56, 64]
    "conv2_out": list(range(16, 129, 16)),   # [16, 32, 48, 64, 80,
96, 112, 128]
    "fc_size": list(range(32, 257, 32))     # [32, 64, 96, 128, 160,
192, 224, 256]
}
```

```
# Create a GridSampler with the defined grid
```

```
grid_sampler = optuna.samplers.GridSampler(param_grid, seed=seed) #
Use seed for reproducibility
```

```
# Create a study object with the GridSampler
```

```
study_grid = optuna.create_study(direction='maximize',
sampler=grid_sampler)
```

```
study_grid.optimize(lambda trial: objective_apples(trial, device),
n_trials=n_trials)
```

```
[I 2026-01-05 09:48:27,252] A new study created in memory with name:
no-name-975d1f79-8400-4dfa-8f22-fbb4aec1848f
```

```
{'healthy': 0, 'rotten': 1}
```

```
{"model_id": "f338f155370b41059a041897b2640846", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "47047692da4f478e841529466d2e5f5c", "version_major": 2, "vers
ion_minor": 0}
```

```
Epoch 5 - Train Loss: 0.3343
```

```
Training complete!
```

```
[I 2026-01-05 09:49:08,597] Trial 0 finished with value:
0.4343891441822052 and parameters: {'conv1_out': 40, 'conv2_out': 112,
'fc_size': 32}. Best is trial 0 with value: 0.4343891441822052.
```

```
{'healthy': 0, 'rotten': 1}
```

```
{"model_id": "a4641e6d545a4ec5bc0a696971442487", "version_major": 2, "vers
ion_minor": 0}
```

```
{"model_id": "57d1404f20c24696927b7c063817583c", "version_major": 2, "vers
ion_minor": 0}
```



Epoch 5 - Train Loss: 0.3459  
Training complete!

[I 2026-01-05 09:49:49,928] Trial 1 finished with value:  
0.44933921098709106 and parameters: {'conv1\_out': 64, 'conv2\_out': 96,  
'fc\_size': 256}. Best is trial 1 with value: 0.44933921098709106.

{'healthy': 0, 'rotten': 1}

{"model\_id": "0b996904cd8245ec81f713a07b725278", "version\_major": 2, "version\_minor": 0}

{"model\_id": "4de52aa379e84711b0db9ae545dc6a4b", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.2648  
Training complete!

[I 2026-01-05 09:50:31,772] Trial 2 finished with value:  
0.7944719791412354 and parameters: {'conv1\_out': 56, 'conv2\_out': 112,  
'fc\_size': 256}. Best is trial 2 with value: 0.7944719791412354.

{'healthy': 0, 'rotten': 1}

{"model\_id": "548fdf94ee0142cca5ced8de169b173a", "version\_major": 2, "version\_minor": 0}

{"model\_id": "f7940049201a4216b293d0514099c7f8", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.2756  
Training complete!

[I 2026-01-05 09:51:14,749] Trial 3 finished with value:  
0.8275692462921143 and parameters: {'conv1\_out': 24, 'conv2\_out': 64,  
'fc\_size': 64}. Best is trial 3 with value: 0.8275692462921143.

{'healthy': 0, 'rotten': 1}

{"model\_id": "eb33d6986c654932afb16b9650fbcdcc", "version\_major": 2, "version\_minor": 0}

{"model\_id": "cb9a1765e1494cbea7796fb782ab5f0f", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3037  
Training complete!

[I 2026-01-05 09:51:56,556] Trial 4 finished with value: 0.8207193613052368 and parameters: {'conv1\_out': 64, 'conv2\_out': 112, 'fc\_size': 64}. Best is trial 3 with value: 0.8275692462921143.

{'healthy': 0, 'rotten': 1}

{"model\_id": "a2e0946048884e6a8d60f4330acaf28f", "version\_major": 2, "version\_minor": 0}

{"model\_id": "991e48e04b9f4c9fb69e1cad25281a34", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.4427  
Training complete!

[I 2026-01-05 09:52:36,995] Trial 5 finished with value: 0.45652174949645996 and parameters: {'conv1\_out': 24, 'conv2\_out': 16, 'fc\_size': 128}. Best is trial 3 with value: 0.8275692462921143.

{'healthy': 0, 'rotten': 1}

{"model\_id": "18b1bf0970de4f93b6a625b5b66fb81d", "version\_major": 2, "version\_minor": 0}

{"model\_id": "3969460b668646b0bc590994c8194d38", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.3234  
Training complete!

[I 2026-01-05 09:53:19,387] Trial 6 finished with value: 0.7238179445266724 and parameters: {'conv1\_out': 32, 'conv2\_out': 32, 'fc\_size': 160}. Best is trial 3 with value: 0.8275692462921143.

{'healthy': 0, 'rotten': 1}

{"model\_id": "e2bce81ce3ee4dcca3a6ea7fe5c2e330", "version\_major": 2, "version\_minor": 0}

{"model\_id": "3cc3dd1da45a4b7e998c7ba536fdb360", "version\_major": 2, "version\_minor": 0}

Epoch 5 - Train Loss: 0.2883  
Training complete!

[I 2026-01-05 09:53:58,926] Trial 7 finished with value: 0.7032498121261597 and parameters: {'conv1\_out': 64, 'conv2\_out': 128, 'fc\_size': 96}. Best is trial 3 with value: 0.8275692462921143.

{'healthy': 0, 'rotten': 1}

```
{"model_id": "0ac242092766486ea05d9c218a4b9c55", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "f910624f8cb740e39715086aa37be698", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 0.3741  
Training complete!

[I 2026-01-05 09:54:39,480] Trial 8 finished with value: 0.5978121161460876 and parameters: {'conv1\_out': 48, 'conv2\_out': 16, 'fc\_size': 192}. Best is trial 3 with value: 0.8275692462921143.

```
{'healthy': 0, 'rotten': 1}
```

```
{"model_id": "f5a4f91da9fb46fbb149f95618862240", "version_major": 2, "version_minor": 0}
```

```
{"model_id": "b32758e123f448eeae299b53d2552b19", "version_major": 2, "version_minor": 0}
```

Epoch 5 - Train Loss: 0.2793  
Training complete!

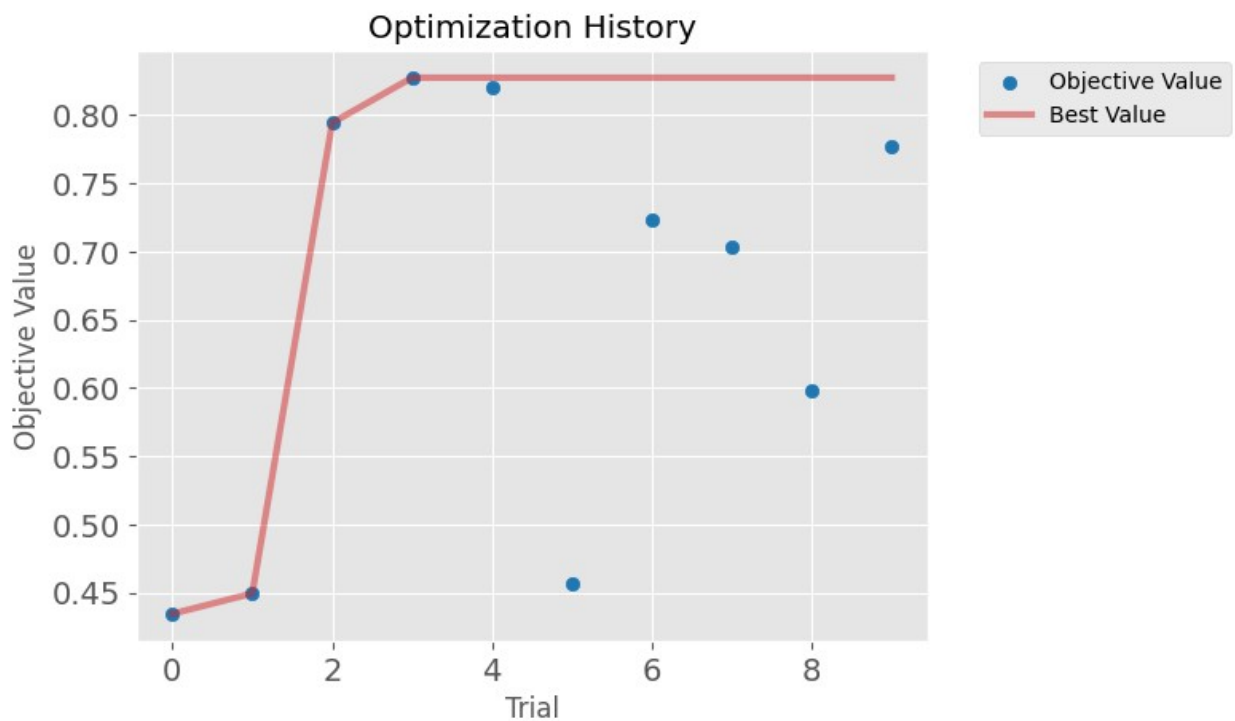
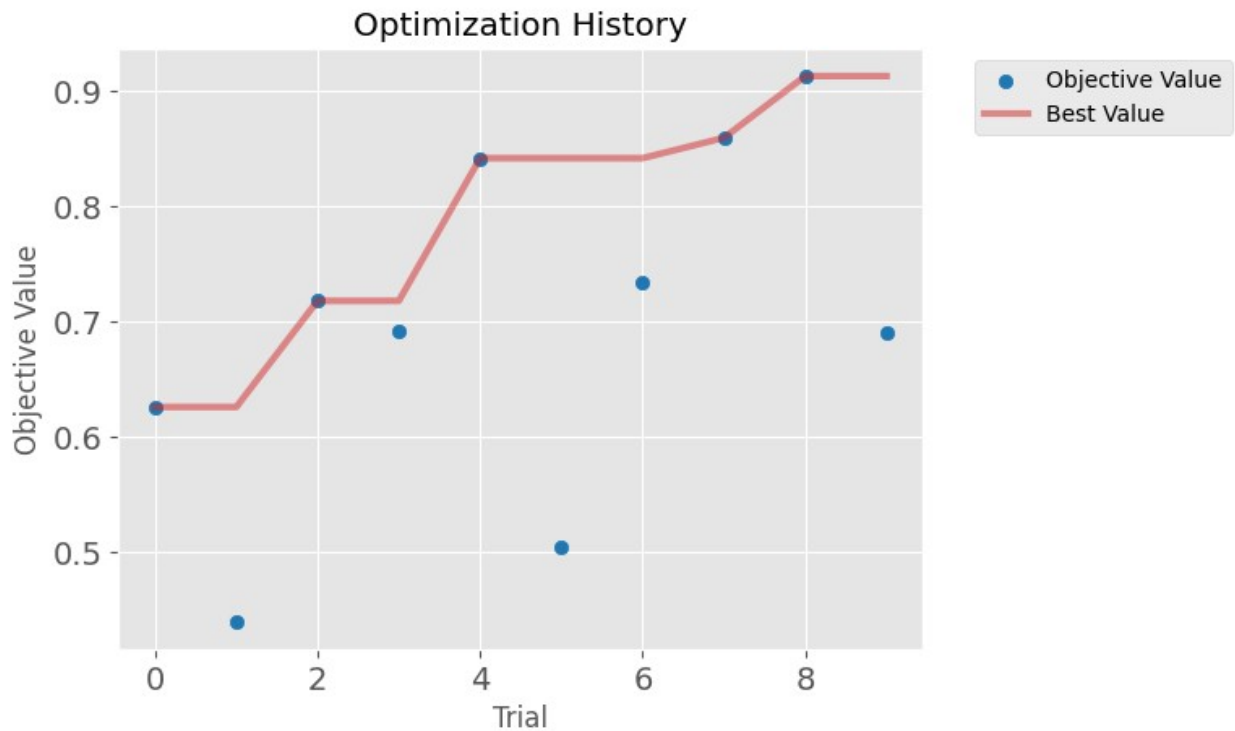
[I 2026-01-05 09:55:17,667] Trial 9 finished with value: 0.7767857313156128 and parameters: {'conv1\_out': 32, 'conv2\_out': 112, 'fc\_size': 256}. Best is trial 3 with value: 0.8275692462921143.

*# Plotting the optimization history*

```
optuna.visualization.matplotlib.plot_optimization_history(study_apples)
plt.title('Optimization History')
plt.show()
```

*# Plotting the optimization history*

```
optuna.visualization.matplotlib.plot_optimization_history(study_grid)
plt.title('Optimization History')
plt.show()
```



Under the grid search approach, the objective values vary significantly because all possible combinations of hyperparameters in the grid are exhaustively evaluated. Since grid search does not take into account the performance of previous trials, it may waste resources evaluating many suboptimal configurations, especially in high-dimensional or redundant hyperparameter spaces.

In contrast, Optuna's search algorithm employs a more intelligent, adaptive sampling strategy (such as TPE). It uses the results of previous trials to guide the search toward more promising regions of the hyperparameter space, allowing for a more efficient and focused exploration. As a result, Optuna is often able to identify better-performing hyperparameter configurations with fewer trials compared to grid search.

The larger and more complex the search space becomes, the more pronounced the advantage of adaptive sampling methods like Optuna over exhaustive methods like grid search.

## Conclusion

Congratulations on completing the hyperparameter optimization tutorial with Optuna!

In this notebook, you explored the key components of an Optuna-based optimization workflow: defining the objective function, setting up the study, and analyzing the results. You also built a flexible CNN architecture designed to accommodate varying hyperparameter configurations, allowing for more dynamic experimentation.

Additionally, you compared Optuna's default search algorithm (TPE) with a traditional grid search, demonstrating the efficiency and adaptability of Optuna in navigating complex hyperparameter spaces. Throughout the tutorial, you evaluated model performance using multiple metrics and learned how to interpret results through Optuna's powerful visualization tools.

By the end of this tutorial, you've gained a practical and conceptual understanding of how to integrate Optuna into deep learning projects, equipping you with a valuable skill for optimizing model performance efficiently and effectively.